Stage 6. Financial

GL chart, account GL/AP defaults, export formats, approval and upload, accruals

PREPARED BY Energy Hippo implementation team
STATUS Ready to enter (revised 2026-11-10)

Stage 6 gives every Northlake bill a GL account to land on and exercises the AP approval-and-upload path Bill Administrators actually use. Most of it is straightforward: a GL chart per company, and a Liability/Expense account plus a Bill Service Type on every billing account from Stage 5. Three pieces are not straightforward, and this chapter says so plainly rather than working around it with SQL: the GL user-defined-fields view is not wired, no AP export format is registered in this installed instance (not even the two the Phase 3 plan called "built-in"), and the accrual reversal has neither a caller nor, in this instance, the table it would write to. Each is entered as far as the product lets it go and then stated as a Finding.

Added 2026-10-01: Ridgeline Wind. The Thermal Plant bought a ten-turbine wind company on July 1, 2024, and Stage 6 gains two entries and one rule because of it. The entries are 5150-WIND, Ridgeline wind energy transfer, in the Thermal Plant's GL chart (Step 1), and the GL defaults of the two Ridgeline billing accounts (Step 3). The rule belongs to the export steps (4 and 5): Ridgeline's monthly transfer statement is GL only, a journal between the plant and its own wind company with no payment leaving the plant, so it must never reach an AP file. The Valley Electric generator account pays only its own small charges and is an ordinary AP account. Its export credit moves to the Central Plant's account under the bill credit transfer, and the credit is coded once, on the Central Plant bill where it lands. Step 2 gains the new code's target values; Step 6 is unchanged.

Before you start

- Stage 5's bills, and the GL account codes Stage 5 assumed (5100-ELEC through 2000-AP), are in place. Step 1's GL accounts should be entered before any Valley Electric or River City bill is imported if you want the importer's Expense code stamped on the imported lines (Step 3, `per_line_coding`).
- The three Ridgeline bills Step 5 uses are in place: the generated July 2024 transfer statement (Stage 7), the generator's first bill (July 1 to August 4, 2024), and the Central Plant's September 2024 bill (August 5 to September 4).
- Read the Findings before Steps 4 and 6 - both describe exactly what is and is not possible to enter today, so you do not spend time looking for a screen that is not there.

Step 1. GL Accounts

Where: Database Administrator, Nodes, the company node's GL Accounts tab.

Enter: 01-gl-accounts.yaml, sixteen accounts across three companies.

Order inside the editor: pick Account Type from the chip dropdown (Assets, Expenses, Liabilities), type the Account Code and Description, and Add. GL accounts are per company - the same code (5100-ELEC) is entered separately in every company that needs it. Northlake University gets all five expense codes plus 2000-AP; Cedar Row Apartments gets four (no Thermal Plant allocation); Northlake Thermal Plant gets its own three utility

codes, 5150-WIND for the Ridgeline Wind transfer, and 2000-AP - it never bills itself the allocation or the solar accounts. Ridgeline bills no other company, so 5150-WIND is entered in the plant's chart only.

Record: nothing generated.

Check: `gl_chart`.

Step 2. GL user-defined fields

Where: target only - see Findings.

Enter: 02-gl-fields.yaml states the five segments (Fund, Department, Program, Activity/Project, Object) and their values per GL account, ready for the day the view exists. The Thermal Plant's list now includes 5150-WIND and a Ridgeline generator override.

Record: nothing; nothing can be typed in today.

Check: none (the underlying columns exist but are unreachable from any screen).

Step 3. Account GL defaults

Where: Database Administrator, Utility Accountant tab, each billing account's Definitions page, Associated G/L Accounts section.

Enter: 03-account-gl-defaults.yaml, Bill Service Type, Bill Entry Type, Liability Account and Default Expense Account for all 51 billing accounts.

Order inside the editor: these four fields sit below the account's own identity fields on the same Definitions page Stage 2 already opened to create the account - there is no separate screen. Liability Account is 2000-AP everywhere, including the negative-control account. Default Expense Account follows the provider (Valley Electric to 5100-ELEC, and so on). Bill Service Type is three real values (No AP Upload, AP Upload Only, Bill Payment - see Findings), not the four the AP/GL plan assumed, and this file now follows Stage 2 and the plan: AP Upload Only for the external utilities and for the six Thermal Plant cost centers, Bill Payment for the three Helios accounts and for Ridgeline's transfer account, No AP Upload for River City's Cedar Row B, the control. The earlier version had the Thermal Plant on Bill Payment and Helios on AP Upload Only, the reverse of Stage 2. Bill Entry Type is Bill Importer for all 31 Valley Electric and River City accounts (Cedar Row's ten included - the earlier version made them Bill Entry), Bill Entry for the 13 Sierra Gas and Helios accounts, Bill Allocation for the six cost centers and Bill Generation for Ridgeline's transfer account.

Per-line coding. `per_line_coding` sets the Expense Account of individual lines (BillItem.BI_EXpAcctID): in the Bill Entry grid for the Sierra Gas estimate and correction and the Helios Science Center February bill, and through the importer's Expense code on each commodity mapping for the Valley Electric and River City lines. The chart has one expense account per provider, so no bill can be mixed-coded; the column works and has nothing different to choose (a finding).

Record: nothing generated.

Check: `account_defaults` (51 accounts), `account_type_counts`, `per_line_coding`.

Step 4. AP and GL export formats

Where: target only - nothing in the product can create a format. Read the Findings first.

Enter: 04-ap-export-formats.yaml now specifies one AP format (`nl_ap_invoice`) and one GL format (`nl_gl_journal`) end to end: the ExportSetup row each must hold (name, log type, builders, batch prefix, file type, status code), the rule for which bills each admits, the file layout (header, detail or balanced debit and credit lines, trailer), the file name pattern (`NLAP_` or `NLGL_` plus a 14-digit timestamp and `.csv`), a run for fiscal 2024-09 with its expected row count, control total, ExportFile and ExportLog rows and status change, and a revert. The earlier seven target formats are replaced by these two (`other_formats_not_specified` says why for each of the rest).

What is on a screen and what is not. On a screen: SysConstants UploadDirAP and uaExportApprovedBills, the page's filter, Approve boxes, Preview and Create. Not on any screen: the ExportSetup rows, the PermissionCatalog and Permissions rows that make the format appear in the picker, the two customer-written builder procedures each format names, and any revert. The only code in the product that writes those is the Massport upgrade scripts. So the definition is written down to be built, no SQL is invented here, and it is a product finding candidate (`finding_candidates`).

Record: nothing; there is nothing to pick on the Preview/Create Interface File screen in this instance.

Check: `export_formats_registered` (BLOCKED while ExportSetup is empty, PASS when both rows exist), then `ap_export_run`, `gl_export_run` and `export_revert`, which are PENDING until a file has been created.

Step 5. AP approval and upload

Where: Web SPA, `#/reporting/APApprovalAndUpload`.

Enter: 05-ap-approval-and-upload.yaml.

Order: filter to a company and period, and select the QC-Approved, AP-eligible bills - everything up to picking a format still works once a format exists and is worth proving. An approved bill with an open validation finding stays out of the queue: the fixture is Sierra Gas Student Center's November 2024 pair, the estimate (935.90, validation tests 3 and 15) and its correction (885.32, tests 15 and 47), whose findings come from the job's Batch Test rules. The job also takes the estimate back to Status_Code 1 after Stage 5 approved it by hand, so the bill is held from the status side and from the queue side. (The earlier example, the Recreation and Aquatics Center's November and December 2024 bills at 5,800.00 and 8,200.00, is dropped: that account is a Valley Electric import and its bills come from the source service.) Two Ridgeline fixtures prove the routing. The generated July 2024 transfer statement (GL only) stays out of the queue even after QC Approval, so no AP file can name it. The generator's first bill (July 1 to August 4,

2024) is an ordinary AP bill for its own charges ($1,077.45), and the bill credit transfer is coded once, on the Central Plant's September 2024 bill. Stop before "Create Interface File" until Step 4's rows and builders exist.

Record: nothing generated.

Check: `blocked_by_validation` (a finding is IsError = 0; IsError = 1 means a check could not run), `ridgeline_gl_only`, `ridgeline_bct_coded_once`.

Step 6. Accruals

Where: Configuration Tool's System Constants (Add Constant), Database Administrator's Commodities (Add Commodity), then `#/BillEntry` for the accrual bill itself. See Findings - the reversal half cannot be entered at all.

Enter: 06-accruals.yaml.

Order: create the Accruals and Reversals commodities for Northlake University first, then point two new System Constants ("MPA Accruals", "MPA Reversals" - the names are inherited from the proc that reads them, not anything Northlake-specific) at their commodity IDs. Then enter the open accrual as an ordinary manual bill on the Accruals commodity with Bill Type Accrual, for the first period Sierra Gas has not yet billed Admin Hall's gas account (2026-01-15 to 2026-02-14). The earlier fixture used Admin Hall's Valley Electric account, a Bill Importer account that Bill Entry cannot open. Bill Type Accrual is what validation test 99 reads; the Accruals commodity is what `rp_OpenAccruals` reads; the AP and GL builders leave Bill Type 2 out.

Record: the two new CY_CommodityIDs, into the System Constants above.

Check: `accrual_pair` (the open half only; the reversal half has no check because it cannot exist).

Stage 6 is done when

- The AP and GL formats are specified end to end and `export_formats_registered` says BLOCKED, or PASS once someone has installed the rows and builders; no file is shown that was never produced. When they exist, `ap_export_run` and `gl_export_run` pass for fiscal 2024-09 and `export_revert` finds no orphan.
- Validation passes and failures are both visible: clean bills reach the AP approval queue, and the deliberate failure (the Student Center pair) stays out of it until its findings clear; no check failed to run (`IsError = 1`).
- The accrual pair is visible exactly as far as the product allows: the open accrual bill (Bill Type Accrual, 3,400.00) exists and appears wherever an open accrual would show up; the reversal does not exist, and nothing pretends otherwise.
- Ridgeline's routing holds: the GL-only transfer statement appears in no AP queue and no AP file, the generator's bill is an ordinary AP bill, and the bill credit transfer is coded once, on the Central Plant bill.
- The 51 accounts carry the Stage 2 values (31 importer, 13 Bill Entry, 6 allocation, 1 generation; 1 No AP Upload, 46 AP Upload Only, 4 Bill Payment).

Run `checks.sql` beside this chapter against the instance database to see each of these as a result set (the rows that cannot be produced yet say BLOCKED or PENDING, never PASS).

Findings

- **The GL user-defined-fields view is not wired.** Confirmed exactly as product-gap-backlog.md item 3 describes: `Models/Companies/ GLUserDefinedModel.cs` and the full API round-trip (`SaveGLAccountFieldAsync/SaveGLAccountFieldValueAsync` on `IGLAccountsDTORepository/IGLAccountsDTOService`, wired through `GLAccountsDTOController`) exist end to end, but no `.xaml` file in the repository references any of them. `GLAccountUDField` itself has no `CompanyID` column - the five segment aliases are one global set, not per-company, which the target file above already assumes. This is the smallest of the three gaps (a view, not a feature) and unblocks the `nl_ap_gludf` export format once built.

- **No AP export format is registered in this installed instance - not even the two the Phase 3 plan called built-in.** `ExportSetup` has zero rows, and `PermissionCatalog` has zero `TypeKey='EX'` rows, in `EEMSuite_Northlake` (both verified by direct query). `Exports_Get` inner-joins the two, so the AP Approval & Upload format picker has nothing to show regardless of which format a user is permitted to run. The `EEMSuite` upgraded-demo database, by contrast, has both catalog rows and both `ExportSetup` rows for `export_ap`/ `export_ap2`, pointing at `HIPPODemo`-schema builder procs (`HIPPODemo.EEM_AP_BillsForApproval_Get`, etc.). That confirms the two "built-in" formats are themselves a demo customer's own customization, not core, always-present product objects - Northlake's install never received the equivalent. Recommend either loading a Northlake-specific pair of `PermissionCatalog`/`ExportSetup` rows (reusing the demo's proc shapes) for the two Northlake formats `04-ap-export-formats.yaml` now defines, or the richer formats product-gap-backlog.md item 2 already scopes.
- **Accruals are blocked twice over, not once.** Product-gap-backlog.md item 4 already says no UI or API calls `rp_OpenAccruals` with `@Reverse=1`. Reading the proc (`Database/EEMSuite/Procs/rp_OpenAccruals.sql`) shows it is worse than unwired: its first executable statement deletes stale rows from `dbo.Accrual_Bill` unconditionally, before it even checks `@Reverse`, and `EEMSuite_Northlake` has no `dbo.Accrual_Bill` table at all (verified against `sys.tables`; only `dbo.BillAccrualMap` exists here). The `EEMSuite` upgraded-demo database has both tables. Calling `rp_OpenAccruals` against Northlake today - with either value of `@Reverse` - fails immediately with "Invalid object name 'Accrual_Bill'". The open accrual itself is unaffected (it is an ordinary manual bill, entered in Step 6); only the reversal path is affected, and it needs the missing table restored before the missing caller is even worth building.
- **Bill Service Type cannot say GL only.** Ridgeline's transfer statement is GL only, and EEM's three Bill Service Type values have no such option; Bill Payment is only the nearest. The Massport AP approval builder (`MPA.MPA_FI_AP_BillsForApproval_Get`) admits `Type_Code in (1, 2)` and reads 2 as both AP and BI, so a Northlake builder that followed it would put a Bill Payment account, the transfer statement included, in the AP file. Northlake's AP builders therefore need an exclusion of their own, and the target file assumes one keyed on Bill Entry Type (Bill Generation). Closing it for real needs GL-only and AP-and-GL values in the lookup, honored by every builder.
- **Nothing in the product creates an export format, its picker row or its revert.** ExportSetup, PermissionCatalog TypeKey EX, the group grant and the two builder procedures are SQL only; Massport's revert is a custom page over `[MPA].[MPA_RevertExports]`, which deletes the history, log and file rows but does not set Bill.Status_Code back. The approval builder receives no fiscal period, so the period of a run is whatever the user filters and ticks. The Northlake definition is in `04-ap-export-formats.yaml`.
- **A validation finding and a failed check are different rows.** In BillValidation_Tbl, IsError = 0 is a finding and IsError = 1 means the check could not run (BillValidation_Run.sql). The earlier `blocked_by_validation` selected IsError = 1 as "open alerts" and so returned nothing while real alerts existed.

None of the above blocks entering what this stage can enter - the GL chart, the account defaults, and the open accrual all go in normally. They are recorded so the AP export and accrual features start from an accurate account of what is actually installed, not what the Phase 3 plan assumed.